

PROJECT CLOSEOUT REPORT

MSBN Transcript Verification System

AI-Powered Nursing Transcript Review Platform

Team	TruScript
University	Mississippi State University
Program	AWS Innovation Hub
Semester	Spring 2026
Client	Mississippi Board of Nursing (MSBN)
Date	May 2026
Deliverable	Proof-of-Concept / Functional Prototype
Live URL	https://dbejm0k2ogvgf.cloudfront.net

PoC Scope Notice

This system was designed and built as a proof-of-concept for the AWS Innovation Hub program. It demonstrates a viable technical approach for automating MSBN transcript verification and is intended to inform future production development. It is not a production-ready system.

Table of Contents

Table of Contents	2
1. Deployment Guide	3
1.1 Prerequisites	3

1. Deployment Guide

This section describes the steps required to deploy the full MSBN Transcript Verification System stack from scratch on AWS. Each step must be completed in order.

1.1 Prerequisites

Ensure the following tools and access are in place before beginning:

- AWS CLI installed and configured with the profile `msaih_IsbAdminsPS` and sufficient IAM permissions
- Docker Desktop running on the deployment machine with support for linux/amd64 cross-platform builds
- Node.js 18 or later and npm installed (for the frontend build)
- Terraform 1.5 or later installed
- The msbn repository cloned locally
- Access to AWS Secrets Manager entries for database and Keycloak admin passwords

1 Infrastructure — Terraform

Navigate to the dev environment configuration directory and apply the Terraform plan. This creates all AWS resources: VPC, subnets, security groups, ECS cluster, RDS instance, ECR repositories, S3 bucket, CloudFront distribution, and ALB.

```
cd infrastructure/environments/dev
terraform init
terraform apply -var-file="terraform.tfvars"
```

Review the plan output before confirming. Terraform will output key resource identifiers (ALB DNS, CloudFront domain, ECR URLs) that are referenced in subsequent steps.

2 Backend API — Build and Deploy

The backend is a Docker image pushed to Amazon ECR and run on ECS Fargate. Because ECS Fargate uses x86 hosts, the image must be built with the linux/amd64 platform flag regardless of the build machine architecture.

Step 1: Authenticate Docker with ECR.

```
aws ecr get-login-password --region us-east-1 --profile msaih_IsbAdminsPS \
| docker login --username AWS --password-stdin \
526958953802.dkr.ecr.us-east-1.amazonaws.com
```

Step 2: Build the backend image. The `--platform linux/amd64` flag is required.

```
cd apps/backend
docker build --platform linux/amd64 -t msbn-backend:latest .
```

Step 3: Tag and push the image to ECR.

```
docker tag msbn-backend:latest \
526958953802.dkr.ecr.us-east-1.amazonaws.com/msbn-backend:latest
```

```
docker push 526958953802.dkr.ecr.us-east-1.amazonaws.com/msbn-backend:latest
```

Step 4: Force ECS to pull the new image and restart the service.

```
aws ecs update-service \
  --cluster msbn-dev-cluster \
  --service msbn-dev-backend \
  --force-new-deployment \
  --region us-east-1 \
  --profile msaih_IsbAdminsPS
```

Wait for the ECS service to reach a stable running state before proceeding. This typically takes 1-3 minutes. Monitor with:

```
aws ecs wait services-stable \
  --cluster msbn-dev-cluster \
  --services msbn-dev-backend \
  --region us-east-1 --profile msaih_IsbAdminsPS
```

3 Keycloak — Deploy and Provision

Keycloak uses the official `quay.io/keycloak/keycloak:24.0` image. It is configured via environment variables set in the ECS task definition. The key configuration values are:

- `KC_DB=postgres`
- `KC_DB_URL=jdbc:postgresql://msbn-dev-postgres.cm16aka82i8r.us-east-1.rds.amazonaws.com:5432/msbn_db?currentSchema=keycloak`
- `KC_DB_USERNAME=msbn_user`
- `KC_DB_PASSWORD` retrieved from AWS Secrets Manager
- Start command: `start-dev --import-realm`

Force a new ECS task deployment:

```
aws ecs update-service \
  --cluster msbn-dev-cluster \
  --service msbn-dev-keycloak \
  --force-new-deployment \
  --region us-east-1 \
  --profile msaih_IsbAdminsPS
```

After Keycloak is running and healthy (check CloudWatch logs at `/ecs/msbn-dev/keycloak` for the Keycloak started message), run the provision script to set user passwords and grant roles. This step is required after every cold start of a fresh Keycloak database.

```
cd apps/backend/keycloak
bash provision-users.sh
```

Important: Provision Script Requirement

The `provision-users.sh` script must be executed after any fresh Keycloak deployment against a clean database. Without it, the seeded users (`staff_user`, `admin_user`) will not have passwords set and the `msbn-api` service account will not have the correct roles. The `POST /api/v1/auth/users` endpoint will return errors until provisioning is complete.

4 Frontend — Build and Deploy

The frontend deployment script handles the full process: npm install, Vite production build, S3 sync, and CloudFront cache invalidation. Run it from the repository root.

```
bash scripts/deploy-frontend.sh dev
```

The script outputs the S3 bucket name, CloudFront distribution ID, invalidation ID, and the live URL. CloudFront propagation typically takes 30-60 seconds after the invalidation is created.

5 Verify the Deployment

After all four steps complete, verify end-to-end functionality:

- Frontend: <https://dbejm0k2ogvgf.cloudfront.net>
- API health check: <https://msbn-dev-alb-2055778124.us-east-1.elb.amazonaws.com/api/v1/health>
- Login as staff: username `staff_user` / password `StaffPass1!`
- Login as admin: username `admin_user` / password `AdminPass1!`
- Confirm the Transcript List loads, navigation works, and the Admin Users page is accessible to the admin account